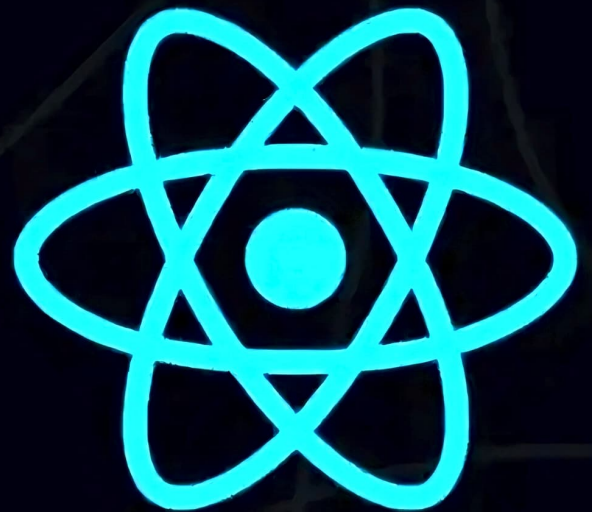


Event Handling





In **React**, handling events is very similar to handling events in **regular JavaScript**. However, there are some syntactic differences and conventions that are important to understand.

Let's go through the basics step by step.

Event Binding

In React, events are written in `camelCase`, and you pass a function as the `event handler`.

```
import React from 'react';

function MyButton() {
  function handleClick() {
    alert('Button was clicked!');
  }

  return (
    <button onClick={handleClick}>
      Click me
    </button>
  );
}

export default MyButton;
```

In this example, we have a `button` element. The `onClick` event is bound to the `handleClick` function. When the button is clicked, the `handleClick` function is called, and an alert message is shown.

Passing Arguments to Event Handlers

Sometimes you might need to pass additional parameters to your event handler.

```
import React from 'react';

function MyButton() {
  function handleClick(name) {
    alert(`Hello, ${name}!`);
  }

  return (
    <button onClick={() => handleClick('React Developer')}>
      Click me
    </button>
  );
}

export default MyButton;
```

In this example, the `handleClick` function takes a parameter name. We use an `arrow function` to pass `'React Developer'` as the argument when the button is clicked.

Preventing Default Behavior

Sometimes you might want to prevent the default behavior of an event.

```
import React from 'react';

function MyForm() {
  function handleSubmit(event) {
    event.preventDefault();
    alert('Form was submitted!');
  }

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Submit</button>
    </form>
  );
}

export default MyForm;
```

In this example, we have a form with a submit button. The `handleSubmit` function calls `event.preventDefault()` to prevent the form from actually submitting, and then shows an alert.

Using State with Events

Often, you'll want to update the state in response to user input.

```
import React, { useState } from 'react';

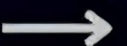
function MyInput() {
  const [text, setText] = useState('');

  function handleChange(event) {
    setText(event.target.value);
  }

  return (
    <div>
      <input type="text" value={text} onChange={handleChange} />
      <p>You typed: {text}</p>
    </div>
  );
}

export default MyInput;
```

In this example, we have an **input field**. The **handleChange** function **updates the text** state whenever the user types something. The value of the input field is **controlled by the state**, and the current value is displayed in a paragraph below the input.



Summary

Event names are camelCase: Use `onClick` instead of `onclick`.

Event handlers are functions: Pass the function reference directly, not as a string.

Prevent default behavior: Use `event.preventDefault()` to stop the default action of the event.

Using state: Update the state using the `useState` hook (in functional components).